

Prodhive — Backend Implementation Plan (Phases 7-13)

Continues from: Phases 1–6 (Organizations, project membership/activity, labels/links/subtasks/ attachments, governed workflow, backlog ranking + sprint lifecycle + analytics)

Ends at: notifications, search/dashboards, GitHub App integration, PR linkage, PR reviews, secure webhooks, and the advanced-features grab bag (configurable workflows, roadmaps, capacity planning, API keys, SSO, import/export).

Same conventions as before: **NEW** = create the file, **MODIFY** = change an existing file. This phase set leans on `IssueStatusHistory` , `ActivityLog` , and `WorkflowService` from Phase 5/6 — make sure those exist before starting Phase 7.

Roadmap

Phase	What it adds	Touches
7	Mentions, notifications, in-app alerts	Prodhive_core
8	Global search, saved filters, dashboards, workload/health	Prodhive_core
9	GitHub App connection, repo mapping, sync	Prodhive_core (new module optional)
10	Issue keys, PR ↔ issue linkage	Prodhive_core
11	PR review tracking + dashboard	Prodhive_core
12	Secure, idempotent GitHub webhooks	Prodhive_core, Prodhive_gateway
13	Configurable workflows, roadmaps, capacity, API keys, SSO (guidance), import/export	Prodhive_core, Prodhive_auth

Phase 7: Collaboration & Notifications

Goal: @mentions in comments, an in-app notification feed, and real-time delivery over the WebSocket connection you already have from Phase 5's board broadcasting.

7a. Notification entity

```
package com.prodhive_core.entity;

import jakarta.persistence.*;
import lombok.*;
import java.time.Instant;

@Entity
@Getter @Setter @NoArgsConstructor @AllArgsConstructor
public class Notification {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false) private Long recipientUserId;
    @Enumerated(EnumType.STRING) @Column(nullable = false)
    private NotificationType type; // MENTIONED, ASSIGNED, STATUS_CHANGED, PR_LINKED, REVIEW_REQUESTED

    @Column(nullable = false, length = 500) private String message;
    @Column(nullable = false) private String entityType; // "ISSUE", "COMMENT", "PULL_REQUEST"
    @Column(nullable = false) private Long entityId;

    @Column(nullable = false) private boolean read = false;
    @Column(nullable = false, updatable = false) private Instant createdAt = Instant.now();
}
// entity/NotificationType.java
// public enum NotificationType { MENTIONED, ASSIGNED, STATUS_CHANGED, PR_LINKED, REVIEW_REQUESTED }
```

```
public interface NotificationRepository extends JpaRepository<Notification, Long> {
    List<Notification> findByRecipientUserIdOrderByCreatedAtDesc(Long userId);
    List<Notification> findByRecipientUserIdAndReadFalse(Long userId);
}
```

7b. NotificationService (NEW)

```

package com.prodhive_core.service;

import com.prodhive_core.entity.*;
import com.prodhive_core.repository.NotificationRepository;
import org.springframework.messaging.simp.SimpMessagingTemplate;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class NotificationService {

    private final NotificationRepository notificationRepository;
    private final SimpMessagingTemplate messagingTemplate;

    public NotificationService(NotificationRepository notificationRepository, SimpMessagingTemplate messagingTemplate) {
        this.notificationRepository = notificationRepository;
        this.messagingTemplate = messagingTemplate;
    }

    public void notify(Long recipientUserId, NotificationType type, String message,
                      String entityType, Long entityId) {
        if (recipientUserId == null) return; // e.g. unassigned issue

        Notification n = new Notification();
        n.setRecipientUserId(recipientUserId);
        n.setType(type);
        n.setMessage(message);
        n.setEntityType(entityType);
        n.setEntityId(entityId);
        Notification saved = notificationRepository.save(n);

        // real-time push to a per-user topic
        messagingTemplate.convertAndSend("/topic/user/" + recipientUserId + "/notifications", saved);
    }

    public List<Notification> myNotifications(Long userId) {
        return notificationRepository.findByRecipientUserIdOrderByCreatedAtDesc(userId);
    }

    public void markRead(Long notificationId) {
        Notification n = notificationRepository.findById(notificationId).orElseThrow();
        n.setRead(true);
        notificationRepository.save(n);
    }

    public void markAllRead(Long userId) {
        notificationRepository.findByRecipientUserIdAndReadFalse(userId)
            .forEach(n -> { n.setRead(true); notificationRepository.save(n); });
    }
}

```

7c. Mentions — parse @username in comments

MODIFY `service/CommentService.java` — inject `NotificationService` and a way to resolve `@username` to a `userId`. Since usernames live in `Prodhive_auth`, the simplest approach without a live cross-service call is: mention by numeric user id in the format `@[123]` inserted by the frontend when someone is selected from an autocomplete (frontend resolves `username` → `id`, then writes the tagged form into the comment body). This avoids a synchronous `Core`→`Auth` network call on every comment.

```

// MODIFY CommentService.create(...)
private static final Pattern MENTION_PATTERN = Pattern.compile("@\\[(\\d+)\\]");

public Comment create(Long issueId, CommentRequest req, Long authorId) {
    Issue issue = issueRepository.findById(issueId).orElseThrow();

    Comment comment = new Comment();
    comment.setBody(req.body());
    comment.setAuthorId(authorId);
    comment.setIssue(issue);
    Comment saved = commentRepository.save(comment);

    Matcher matcher = MENTION_PATTERN.matcher(req.body());
    while (matcher.find()) {
        Long mentionedUserId = Long.valueOf(matcher.group(1));
        if (!mentionedUserId.equals(authorId)) {
            notificationService.notify(mentionedUserId, NotificationType.MENTIONED,
                "You were mentioned on " + issue.getIssueKey(), "COMMENT", saved.getId());
        }
    }

    activityLogService.log(issue.getBoard().getProject().getId(), "ISSUE", issue.getId(),
        "COMMENTED", authorId, null);

    return saved;
}

```

Rendering `@[123]` back into a readable "@Jane Doe" is a frontend concern — the frontend already has the project's member list (from Phase 3's `ProjectMember` endpoints) to resolve IDs to names client-side, so no extra backend work is needed there.

7d. Wire notifications into existing flows

MODIFY `service/IssueService.java` — in `assign()` and `updateStatus()` :

```
// in assign(id, assigneeId):
notificationService.notify(assigneeId, NotificationType.ASSIGNED,
    "You were assigned to " + issue.getIssueKey(), "ISSUE", issue.getId());

// in updateStatus(...), after saving:
if (issue.getAssigneeId() != null && !issue.getAssigneeId().equals(changedBy)) {
    notificationService.notify(issue.getAssigneeId(), NotificationType.STATUS_CHANGED,
        issue.getIssueKey() + " moved to " + newStatus, "ISSUE", issue.getId());
}
```

7e. NotificationController (NEW)

```
package com.prodhive_core.controller;

import com.prodhive_core.entity.Notification;
import com.prodhive_core.security.CurrentUser;
import com.prodhive_core.service.NotificationService;
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/core/notifications")
public class NotificationController {

    private final NotificationService notificationService;
    private final CurrentUser currentUser;

    public NotificationController(NotificationService notificationService, CurrentUser currentUser) {
        this.notificationService = notificationService;
        this.currentUser = currentUser;
    }

    @GetMapping
    public ResponseEntity<List<Notification>> list(HttpServletRequest http) {
        return ResponseEntity.ok(notificationService.myNotifications(currentUser.getUserId(http)));
    }

    @PatchMapping("/{id}/read")
    public ResponseEntity<Void> markRead(@PathVariable Long id) {
        notificationService.markRead(id);
        return ResponseEntity.ok().build();
    }

    @PatchMapping("/read-all")
    public ResponseEntity<Void> markAllRead(HttpServletRequest http) {
        notificationService.markAllRead(currentUser.getUserId(http));
        return ResponseEntity.ok().build();
    }
}
```

Email delivery is still not implemented (same gap as Phase 2). The WebSocket push above gives you real in-app alerts, which is the more demo-visible half anyway. If you want actual emails, that's a Spring Mail + SMTP provider addition — worth naming as scoped-out infrastructure in your report rather than attempting it under time pressure.

✓ **Phase 7 done when:** mentioning a user in a comment creates a notification they receive live (open two browser sessions, mention one user from the other), assigning/moving an issue notifies the assignee, and `GET /api/core/notifications` + mark-read endpoints work.

Phase 8: Search & Dashboards

Goal: a global search box, saved filters, and the aggregate endpoints your Dashboard page (currently a static mockup) needs to become real.

8a. Global search

```
// MODIFY repository/IssueRepository.java — add:
@Query("SELECT i FROM Issue i WHERE i.board.project.id = :projectId AND " +
    "(LOWER(i.title) LIKE LOWER(CONCAT('%', :q, '%')) OR " +
    " LOWER(i.description) LIKE LOWER(CONCAT('%', :q, '%')) OR " +
    " LOWER(i.issueKey) LIKE LOWER(CONCAT('%', :q, '%')))"
List<Issue> search(@Param("projectId") Long projectId, @Param("q") String query);
```

```
// controller/SearchController.java (NEW)
package com.prodhive_core.controller;

import com.prodhive_core.entity.Issue;
import com.prodhive_core.repository.IssueRepository;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/core/search")
public class SearchController {

    private final IssueRepository issueRepository;

    public SearchController(IssueRepository issueRepository) {
        this.issueRepository = issueRepository;
    }

    @GetMapping
    public ResponseEntity<List<Issue>> search(@RequestParam Long projectId, @RequestParam String q) {
        return ResponseEntity.ok(issueRepository.search(projectId, q));
    }
}
```

A **LIKE** query is fine at student-project data volume. Real full-text search (Postgres `tsvector` / `GIN` index, or Elasticsearch) is the "correct at scale" answer — worth one sentence in your report as a known simplification, not worth implementing under time pressure.

8b. Saved filters

```
// entity/SavedFilter.java
package com.prodhive_core.entity;
import jakarta.persistence.*;
import lombok.*;

@Entity
@Getter @Setter @NoArgsConstructor @AllArgsConstructor
public class SavedFilter {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false) private Long userId;
    @Column(nullable = false) private Long projectId;
    @Column(nullable = false) private String name;
    @Column(nullable = false, length = 1000) private String filterJson; // e.g. {"assigneeId":5,"priority":"HIGH"}
}
```

```
public interface SavedFilterRepository extends JpaRepository<SavedFilter, Long> {
    List<SavedFilter> findByUserIdAndProjectId(Long userId, Long projectId);
}

// controller/SavedFilterController.java — straightforward CRUD under
// /api/core/projects/{projectId}/saved-filters, same pattern as LabelController from Phase 4.
```

8c. Dashboard aggregate endpoints

MODIFY `repository/IssueRepository.java` — add:

```
List<Issue> findByAssigneeId(Long assigneeId);
```

```
// controller/DashboardController.java (NEW)
package com.prodhive_core.controller;

import com.prodhive_core.entity.*;
import com.prodhive_core.repository.*;
import com.prodhive_core.security.CurrentUser;
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/api/core/dashboard")
public class DashboardController {

    private final IssueRepository issueRepository;
    private final ActivityLogRepository activityLogRepository;
    private final ProjectMemberRepository projectMemberRepository;
    private final CurrentUser currentUser;

    public DashboardController(IssueRepository issueRepository, ActivityLogRepository activityLogRepository,
                              ProjectMemberRepository projectMemberRepository, CurrentUser currentUser) {
        this.issueRepository = issueRepository;
        this.activityLogRepository = activityLogRepository;
        this.projectMemberRepository = projectMemberRepository;
        this.currentUser = currentUser;
    }

    @GetMapping("/my-issues")
    public ResponseEntity<List<Issue>> myIssues(HttpServletRequest http) {
        return ResponseEntity.ok(issueRepository.findByAssigneeId(currentUser.getUserId(http)));
    }

    @GetMapping("/recent-activity")
    public ResponseEntity<List<ActivityLog>> recentActivity(HttpServletRequest http) {
        Long userId = currentUser.getUserId(http);
        List<Long> myProjectIds = projectMemberRepository.findById(null); // placeholder — see note below
        // Practical approach: add a derived method
        // ActivityLogRepository.findByIdInOrderByCreatedAtDesc(List<Long> projectIds)
        // where projectIds comes from a new ProjectMemberRepository.findProjectIdsByUserId(userId) query.
        return ResponseEntity.ok(List.of()); // replace with real query per the note
    }
}
```

The `recentActivity` stub above needs one more repository method: `@Query("SELECT pm.project.id FROM ProjectMember pm WHERE pm.userId = :userId") List<Long> findProjectIdsByUserId(@Param("userId") Long userId)` on `ProjectMemberRepository`, then `ActivityLogRepository.findByIdInOrderByCreatedAtDesc(projectIds)` — shown separately here so the placeholder above doesn't silently compile-fail if copied verbatim.

8d. Workload & project health

```
// service/WorkloadService.java (NEW)
package com.prodhive_core.service;

import com.prodhive_core.entity.Issue;
import com.prodhive_core.entity.IssueStatus;
import com.prodhive_core.repository.IssueRepository;
import org.springframework.stereotype.Service;
import java.util.*;
import java.util.stream.Collectors;

@Service
public class WorkloadService {

    private final IssueRepository issueRepository;

    public WorkloadService(IssueRepository issueRepository) {
        this.issueRepository = issueRepository;
    }

    // open issue count per assignee, for a board
    public Map<Long, Long> workloadByAssignee(Long boardId) {
        return issueRepository.findByBoardId(boardId).stream()
            .filter(i -> i.getStatus() != IssueStatus.DONE && i.getAssigneeId() != null)
            .collect(Collectors.groupingBy(Issue::getAssigneeId, Collectors.counting()));
    }

    // simple 0-100 health score: % done, penalized by overdue-looking stale issues
    public int projectHealthScore(Long boardId) {
        List<Issue> issues = issueRepository.findByBoardId(boardId);
        if (issues.isEmpty()) return 100;
        long done = issues.stream().filter(i -> i.getStatus() == IssueStatus.DONE).count();
        double doneRatio = (double) done / issues.size();
        long stale = issues.stream()
            .filter(i -> i.getStatus() == IssueStatus.IN_PROGRESS)
            .filter(i -> i.getUpdatedAt().isBefore(java.time.Instant.now().minus(7, java.time.temporal.ChronoUnit.DAYS)))
            .count();
        int score = (int) (doneRatio * 100) - (int) (stale * 5);
        return Math.max(0, Math.min(100, score));
    }
}
```

Expose both via a small `WorkloadController` (`GET /api/core/boards/{id}/workload` , `GET /api/core/boards/{id}/health`) — same controller pattern used throughout.

✓ **Phase 8 done when:** search returns matching issues by title/description/key, saved filters persist and reload, `/dashboard/my-issues` returns real assigned issues, and workload/ health numbers change sensibly as you move issues around.

Phase 9: GitHub Integration

Goal: connect a project to a GitHub repo via OAuth, store the mapping, and pull basic repo data. This phase is genuinely the most infrastructure-heavy one — a real GitHub App requires a registered app with a client ID/secret and a public callback URL, which needs upfront setup outside your code.

Scope decision to make now: a full "GitHub App installation" flow (what your plan names) means registering a GitHub App in GitHub's developer settings, handling the App's installation webhook, and managing installation-scoped tokens. A simpler, fully sufficient alternative for a student project is a **GitHub OAuth App** (or even a personal access token pasted in by the PM) that authorizes read access to one repo. This guide implements the simpler OAuth/PAT version — name the full GitHub App flow as "future work: GitHub App marketplace listing" in your report if asked why you didn't build the heavier version.

9a. GitHubIntegration entity

```
package com.prodhive_core.entity;

import jakarta.persistence.*;
import lombok.*;
import java.time.Instant;

@Entity
@Getter @Setter @NoArgsConstructor @AllArgsConstructor
public class GitHubIntegration {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne @JoinColumn(name = "project_id", nullable = false)
    private Project project;

    @Column(nullable = false) private String repoFullName; // "org/repo"
    @Column(nullable = false, length = 500) private String accessToken; // encrypt at rest in real deployment
    @Column(nullable = false) private Long connectedBy;
    @Column(nullable = false, updatable = false) private Instant connectedAt = Instant.now();
}
```

Storing a raw access token in plain text in the database is a real security gap — for a demo it's common enough to be forgivable, but say so explicitly in your report rather than presenting it as production-ready. A real fix is field-level encryption (e.g. Jasypt) or a secrets manager.

```
public interface GitHubIntegrationRepository extends JpaRepository<GitHubIntegration, Long> {
    Optional<GitHubIntegration> findByProjectId(Long projectId);
    Optional<GitHubIntegration> findByRepoFullName(String repoFullName);
}
```

9b. GitHubApiClient — thin wrapper over GitHub's REST API

```
package com.prodhive_core.integration;

import com.fasterxml.jackson.databind.JsonNode;
import org.springframework.http.*;
import org.springframework.stereotype.Component;
import org.springframework.web.client.RestTemplate;

@Component
public class GitHubApiClient {

    private final RestTemplate restTemplate = new RestTemplate();
    private static final String BASE_URL = "https://api.github.com";

    public JsonNode getRepo(String repoFullName, String accessToken) {
        return get("/repos/" + repoFullName, accessToken);
    }

    public JsonNode listPullRequests(String repoFullName, String accessToken) {
        return get("/repos/" + repoFullName + "/pulls?state=all", accessToken);
    }

    private JsonNode get(String path, String accessToken) {
        HttpHeaders headers = new HttpHeaders();
        headers.set("Authorization", "Bearer " + accessToken);
        headers.set("Accept", "application/vnd.github+json");
        HttpEntity<Void> entity = new HttpEntity<>(headers);
        ResponseEntity<JsonNode> response = restTemplate.exchange(
            BASE_URL + path, HttpMethod.GET, entity, JsonNode.class);
        return response.getBody();
    }
}
```

9c. Connect + sync endpoints

```
// controller/GitHubIntegrationController.java (NEW)
package com.prodhive_core.controller;

import com.fasterxml.jackson.databind.JsonNode;
import com.prodhive_core.entity.GitHubIntegration;
import com.prodhive_core.integration.GitHubApiClient;
import com.prodhive_core.repository.GitHubIntegrationRepository;
import com.prodhive_core.repository.ProjectRepository;
import com.prodhive_core.security.CurrentUser;
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/core/projects/{projectId}/github")
public class GitHubIntegrationController {

    private final GitHubIntegrationRepository integrationRepository;
    private final ProjectRepository projectRepository;
    private final GitHubApiClient gitHubApiClient;
    private final CurrentUser currentUser;

    public GitHubIntegrationController(GitHubIntegrationRepository integrationRepository,
                                      ProjectRepository projectRepository,
                                      GitHubApiClient gitHubApiClient, CurrentUser currentUser) {
        this.integrationRepository = integrationRepository;
        this.projectRepository = projectRepository;
        this.gitHubApiClient = gitHubApiClient;
        this.currentUser = currentUser;
    }

    // simplest connect flow: PM pastes a GitHub personal access token + repo name from the UI
    @PostMapping("/connect")
    public ResponseEntity<GitHubIntegration> connect(@PathVariable Long projectId,
                                                    @RequestBody ConnectRequest req,
                                                    HttpServletRequest http) {
        GitHubIntegration integration = new GitHubIntegration();
        integration.setProject(projectRepository.findById(projectId).orElseThrow());
        integration.setRepoFullName(req.repoFullName());
        integration.setAccessToken(req.accessToken());
        integration.setConnectedBy(currentUser.getUserId(http));
        return ResponseEntity.ok(integrationRepository.save(integration));
    }

    @GetMapping("/sync")
    public ResponseEntity<JsonNode> sync(@PathVariable Long projectId) {
        GitHubIntegration integration = integrationRepository.findById(projectId).orElseThrow();
        return ResponseEntity.ok(gitHubApiClient.listPullRequests(
            integration.getRepoFullName(), integration.getAccessToken()));
    }
}

// dto: record ConnectRequest(String repoFullName, String accessToken) {}
```

A real GitHub App OAuth flow (redirect → GitHub consent screen → callback with a code → exchange for a token) is a bigger lift needing a registered OAuth App with client ID/secret and a public callback URL. If you want it: register an OAuth App at github.com/settings/developers, add `spring-boot-starter-oauth2-client`, and configure it as a standard Spring Security OAuth2 client — that's a well-trodden path with lots of documentation, but treat it as a stretch addition on top of the PAT version above, not a blocker to finishing this phase.

✓ **Phase 9 done when:** a PM can paste a repo name + access token, it's stored, and `GET .../github/sync` returns a real list of that repo's pull requests from the GitHub API.

Phase 10: Pull Request Linkage

Goal: issues get a human-readable key (a prerequisite you don't have yet), and PRs get automatically associated with issues by parsing branch names / PR titles / commit messages.

10a. Issue keys (prerequisite — add this first)

MODIFY `entity/Project.java` — add:

```
@Column(nullable = false)
private Integer issueSequence = 0;
```

MODIFY `entity/Issue.java` — add:

```
@Column(nullable = false, unique = true)
private String issueKey; // e.g. "PROD-15"
```

MODIFY `service/IssueService.create(...)` — generate the key atomically:

```
Project project = board.getProject();
project.setIssueSequence(project.getIssueSequence() + 1);
projectRepository.save(project);
issue.setIssueKey(project.getKey() + "-" + project.getIssueSequence());
```

MODIFY `repository/IssueRepository.java` — add:

```
Optional<Issue> findByIssueKey(String issueKey);
```

10b. PullRequest entity

```
package com.prodhive_core.entity;

import jakarta.persistence.*;
import lombok.*;
import java.time.Instant;

@Entity
@Getter @Setter @NoArgsConstructor @AllArgsConstructor
public class PullRequest {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne @JoinColumn(name = "issue_id", nullable = false)
    private Issue issue;

    @Column(nullable = false) private Integer prNumber;
    @Column(nullable = false) private String repoFullName;
    @Column(nullable = false) private String prUrl;
    @Column(nullable = false) private String title;
    @Column(nullable = false) private String authorUsername;
    @Column(nullable = false) private String branchName;

    @Enumerated(EnumType.STRING) @Column(nullable = false)
    private PrStatus status = PrStatus.OPEN;

    @Column(nullable = false, updatable = false) private Instant createdAt = Instant.now();
    private Instant mergedAt;
}
// entity/PrStatus.java — public enum PrStatus { OPEN, MERGED, CLOSED }
```

```
public interface PullRequestRepository extends JpaRepository<PullRequest, Long> {
    List<PullRequest> findByIssueId(Long issueId);
    Optional<PullRequest> findByRepoFullNameAndPrNumber(String repoFullName, Integer prNumber);
}
```

10c. PrLinkingService — the actual matching logic

```

package com.prodhive_core.service;

import com.prodhive_core.entity.*;
import com.prodhive_core.repository.IssueRepository;
import com.prodhive_core.repository.PullRequestRepository;
import org.springframework.stereotype.Service;
import java.util.Optional;
import java.util.regex.*;

@Service
public class PrLinkingService {

    private final IssueRepository issueRepository;
    private final PullRequestRepository pullRequestRepository;
    private final IssueService issueService;
    private final NotificationService notificationService;

    private static final Pattern ISSUE_KEY_PATTERN = Pattern.compile("[A-Z]+-\\d+");

    public PrLinkingService(IssueRepository issueRepository, PullRequestRepository pullRequestRepository,
        IssueService issueService, NotificationService notificationService) {
        this.issueRepository = issueRepository;
        this.pullRequestRepository = pullRequestRepository;
        this.issueService = issueService;
        this.notificationService = notificationService;
    }

    // called by the webhook handler (Phase 12) and by manual linking (10d)
    public Optional<PullRequest> linkFromText(String title, String branchName, String repoFullName,
        int prNumber, String prUrl, String author, boolean merged,
        String action) {

        String candidateText = title + " " + branchName;
        Matcher matcher = ISSUE_KEY_PATTERN.matcher(candidateText.toUpperCase());
        if (!matcher.find()) return Optional.empty();

        String issueKey = matcher.group(1);
        Issue issue = issueRepository.findByIssueKey(issueKey).orElse(null);
        if (issue == null) return Optional.empty();

        PullRequest pr = pullRequestRepository.findByRepoFullNameAndPrNumber(repoFullName, prNumber)
            .orElseGet(PullRequest::new);
        pr.setIssue(issue);
        pr.setPrNumber(prNumber);
        pr.setRepoFullName(repoFullName);
        pr.setPrUrl(prUrl);
        pr.setTitle(title);
        pr.setBranchName(branchName);
        pr.setAuthorUsername(author);

        applyStatusSideEffects(issue, action, merged);

        if (merged) { pr.setStatus(PrStatus.MERGED); pr.setMergedAt(java.time.Instant.now()); }
        else if ("closed".equals(action)) pr.setStatus(PrStatus.CLOSED);
        else pr.setStatus(PrStatus.OPEN);

        PullRequest saved = pullRequestRepository.save(pr);

        if (issue.getReporterId() != null) {
            notificationService.notify(issue.getReporterId(), NotificationType.PR_LINKED,
                "PR #" + prNumber + " linked to " + issue.getIssueKey(), "PULL_REQUEST", saved.getId());
        }
        return Optional.of(saved);
    }

    private void applyStatusSideEffects(Issue issue, String action, boolean merged) {
        if (("opened".equals(action) || "reopened".equals(action)) && issue.getStatus() == IssueStatus.IN_PROGRESS) {
            issueService.updateStatus(issue.getId(), "IN_REVIEW", issue.getReporterId());
        } else if ("closed".equals(action) && merged && issue.getStatus() == IssueStatus.IN_REVIEW) {
            issueService.updateStatus(issue.getId(), "DONE", issue.getReporterId());
        }
    }
}

```

10d. Manual linking fallback (useful before webhooks/Phase 12 are live)

```
// New endpoint on IssueController:
@PostMapping("/{id}/pull-requests")
public ResponseEntity<PullRequest> linkManually(@PathVariable Long id, @RequestBody ManualLinkRequest req) {
    return ResponseEntity.ok(prLinkingService.linkFromText(
        req.title(), req.branchName(), req.repoFullName(), req.prNumber(),
        req.prUrl(), req.author(), false, "opened"
    ).orElseThrow(() -> new IllegalArgumentException("No issue key found in title/branch")));
}

// dto: record ManualLinkRequest(String title, String branchName, String repoFullName,
//                                Integer prNumber, String prUrl, String author) {}

@GetMapping("/{id}/pull-requests")
public ResponseEntity<List<PullRequest>> listPrs(@PathVariable Long id) {
    return ResponseEntity.ok(pullRequestRepository.findByIssueId(id));
}
```

✓ **Phase 10 done when:** creating an issue produces a key like `PROD-1` , manually POSTing a PR whose title contains that key links it and moves the issue to `IN_REVIEW`, and `GET .../pull-requests` returns it.

Phase 11: Pull Request Reviews

Goal: track review state (approved/changes requested) per PR and surface a "needs my review" dashboard.

11a. Review entity

```
package com.prodhive_core.entity;

import jakarta.persistence.*;
import lombok.*;
import java.time.Instant;

@Entity
@Getter @Setter @NoArgsConstructor @AllArgsConstructor
public class Review {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne @JoinColumn(name = "pull_request_id", nullable = false)
    private PullRequest pullRequest;

    @Column(nullable = false) private String reviewerUsername;
    @Enumerated(EnumType.STRING) @Column(nullable = false)
    private ReviewState state; // APPROVED, CHANGES_REQUESTED, COMMENTED

    @Column(length = 2000) private String body;
    @Column(nullable = false) private Instant submittedAt;
}
// entity/ReviewState.java — public enum ReviewState { APPROVED, CHANGES_REQUESTED, COMMENTED }
```

```
public interface ReviewRepository extends JpaRepository<Review, Long> {
    List<Review> findByPullRequestId(Long pullRequestId);
}
```

11b. ReviewService — ingest from webhook (wired fully in Phase 12)

```
package com.prodhive_core.service;

import com.prodhive_core.entity.*;
import com.prodhive_core.repository.PullRequestRepository;
import com.prodhive_core.repository.ReviewRepository;
import org.springframework.stereotype.Service;
import java.time.Instant;

@Service
public class ReviewService {

    private final ReviewRepository reviewRepository;
    private final PullRequestRepository pullRequestRepository;
    private final NotificationService notificationService;

    public ReviewService(ReviewRepository reviewRepository, PullRequestRepository pullRequestRepository,
        NotificationService notificationService) {
        this.reviewRepository = reviewRepository;
        this.pullRequestRepository = pullRequestRepository;
        this.notificationService = notificationService;
    }

    public Review recordReview(Long pullRequestId, String reviewerUsername, String state, String body) {
        PullRequest pr = pullRequestRepository.findById(pullRequestId).orElseThrow();

        Review review = new Review();
        review.setPullRequest(pr);
        review.setReviewerUsername(reviewerUsername);
        review.setState(ReviewState.valueOf(state.toUpperCase()));
        review.setBody(body);
        review.setSubmittedAt(Instant.now());
        Review saved = reviewRepository.save(review);

        if (review.getState() == ReviewState.CHANGES_REQUESTED && pr.getIssue().getReporterId() != null) {
            notificationService.notify(pr.getIssue().getReporterId(), NotificationType.REVIEW_REQUESTED,
                reviewerUsername + " requested changes on PR #" + pr.getPrNumber(),
                "PULL_REQUEST", pr.getId());
        }

        return saved;
    }
}
```

11c. Code review dashboard endpoint

```
// controller/ReviewController.java (NEW)
package com.prodhive_core.controller;

import com.prodhive_core.entity.Review;
import com.prodhive_core.repository.ReviewRepository;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/core/pull-requests/{prId}/reviews")
public class ReviewController {

    private final ReviewRepository reviewRepository;

    public ReviewController(ReviewRepository reviewRepository) {
        this.reviewRepository = reviewRepository;
    }

    @GetMapping
    public ResponseEntity<List<Review>> list(@PathVariable Long prId) {
        return ResponseEntity.ok(reviewRepository.findByPullRequestId(prId));
    }
}
```

A per-project "PRs awaiting review" view can be a simple derived query added to `PullRequestRepository` : `List<PullRequest> findByStatusAndIssue_Board_Project_Id(PrStatus status, Long projectId)` filtered to `OPEN` , exposed via a small `GET /api/core/projects/{id}/pull-requests/pending` endpoint — same pattern as everything above, omitted here for space.

✓ **Phase 11 done when:** a review recorded against a PR shows up under `GET .../reviews` , and a `CHANGES_REQUESTED` review notifies the issue's reporter.

Phase 12: GitHub Webhooks (secure, idempotent)

Goal: the real trigger for Phase 10/11 — a webhook endpoint that verifies GitHub's signature, ignores duplicate deliveries, and logs everything for audit, instead of the naive unauthenticated version shown in earlier drafts.

12a. WebhookDelivery — dedup + audit log

```
package com.prodhive_core.entity;

import jakarta.persistence.*;
import lombok.*;
import java.time.Instant;

@Entity
@Getter @Setter @NoArgsConstructor @AllArgsConstructor
public class WebhookDelivery {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true) private String deliveryId; // X-GitHub-Delivery header
    @Column(nullable = false) private String eventType; // X-GitHub-Event header
    @Column(nullable = false) private String status; // RECEIVED, PROCESSED, FAILED
    @Column(length = 4000) private String errorMessage;
    @Column(nullable = false, updatable = false) private Instant receivedAt = Instant.now();
}

public interface WebhookDeliveryRepository extends JpaRepository<WebhookDelivery, Long> {
    Optional<WebhookDelivery> findByDeliveryId(String deliveryId);
}
```

12b. Signature verification

```
package com.prodhive_core.security;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;
import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;
import java.nio.charset.StandardCharsets;

@Component
public class GitHubSignatureVerifier {

    @Value("${github.webhook.secret}")
    private String webhookSecret;

    public boolean isValid(String payload, String signatureHeader) {
        if (signatureHeader == null || !signatureHeader.startsWith("sha256=")) return false;
        try {
            Mac mac = Mac.getInstance("HmacSHA256");
            mac.init(new SecretKeySpec(webhookSecret.getBytes(StandardCharsets.UTF_8), "HmacSHA256"));
            byte[] hash = mac.doFinal(payload.getBytes(StandardCharsets.UTF_8));
            String computed = "sha256=" + bytesToHex(hash);
            return java.security.MessageDigest.isEqual(
                computed.getBytes(StandardCharsets.UTF_8),
                signatureHeader.getBytes(StandardCharsets.UTF_8));
        } catch (Exception e) {
            return false;
        }
    }

    private String bytesToHex(byte[] bytes) {
        StringBuilder sb = new StringBuilder();
        for (byte b : bytes) sb.append(String.format("%02x", b));
        return sb.toString();
    }
}
```

MODIFY `application.yml` (Prodhive_core) — add:

```
github:
  webhook:
    secret: ${GITHUB_WEBHOOK_SECRET:set-this-in-your-github-webhook-config}
```

12c. Webhook controller — verify, dedup, respond fast, process async

```

package com.prodhive_core.controller;

import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.prodhive_core.entity.WebhookDelivery;
import com.prodhive_core.repository.WebhookDeliveryRepository;
import com.prodhive_core.security.GitHubSignatureVerifier;
import com.prodhive_core.service.GitHubWebhookProcessingService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/core/webhooks/github")
public class GitHubWebhookController {

    private final GitHubSignatureVerifier signatureVerifier;
    private final WebhookDeliveryRepository deliveryRepository;
    private final GitHubWebhookProcessingService processingService;
    private final ObjectMapper objectMapper = new ObjectMapper();

    public GitHubWebhookController(GitHubSignatureVerifier signatureVerifier,
                                   WebhookDeliveryRepository deliveryRepository,
                                   GitHubWebhookProcessingService processingService) {
        this.signatureVerifier = signatureVerifier;
        this.deliveryRepository = deliveryRepository;
        this.processingService = processingService;
    }

    @PostMapping
    public ResponseEntity<String> receive(@RequestHeader("X-GitHub-Event") String eventType,
                                           @RequestHeader("X-GitHub-Delivery") String deliveryId,
                                           @RequestHeader("X-Hub-Signature-256") String signature,
                                           @RequestBody String rawPayload) {

        if (!signatureVerifier.isValid(rawPayload, signature)) {
            return ResponseEntity.status(401).body("invalid signature");
        }

        if (deliveryRepository.findByDeliveryId(deliveryId).isPresent()) {
            return ResponseEntity.ok("duplicate delivery ignored"); // idempotency
        }

        WebhookDelivery delivery = new WebhookDelivery();
        delivery.setDeliveryId(deliveryId);
        delivery.setEventType(eventType);
        delivery.setStatus("RECEIVED");
        deliveryRepository.save(delivery);

        try {
            JsonNode payload = objectMapper.readTree(rawPayload);
            processingService.processAsync(delivery.getId(), eventType, payload);
        } catch (Exception e) {
            delivery.setStatus("FAILED");
            delivery.setErrorMessage(e.getMessage());
            deliveryRepository.save(delivery);
            return ResponseEntity.status(500).body("processing error");
        }

        return ResponseEntity.ok("accepted");
    }
}

```

12d. Async processing so GitHub doesn't time out and retry unnecessarily

```

package com.prodhive_core.service;

import com.fasterxml.jackson.databind.JsonNode;
import com.prodhive_core.entity.WebhookDelivery;
import com.prodhive_core.repository.WebhookDeliveryRepository;
import org.springframework.scheduling.annotation.Async;
import org.springframework.stereotype.Service;

@Service
public class GitHubWebhookProcessingService {

    private final PrLinkingService prLinkingService;
    private final ReviewService reviewService;
    private final WebhookDeliveryRepository deliveryRepository;

    public GitHubWebhookProcessingService(PrLinkingService prLinkingService, ReviewService reviewService,
                                         WebhookDeliveryRepository deliveryRepository) {
        this.prLinkingService = prLinkingService;
        this.reviewService = reviewService;
        this.deliveryRepository = deliveryRepository;
    }

    @Async
    public void processAsync(Long deliveryDbId, String eventType, JsonNode payload) {
        WebhookDelivery delivery = deliveryRepository.findById(deliveryDbId).orElseThrow();
        try {
            switch (eventType) {
                case "pull_request" -> handlePullRequest(payload);
                case "pull_request_review" -> handleReview(payload);
                default -> { /* ignore other event types */ }
            }
            delivery.setStatus("PROCESSED");
        } catch (Exception e) {
            delivery.setStatus("FAILED");
            delivery.setErrorMessage(e.getMessage());
        }
        deliveryRepository.save(delivery);
    }

    private void handlePullRequest(JsonNode payload) {
        String action = payload.get("action").asText();
        JsonNode pr = payload.get("pull_request");
        prLinkingService.linkFromText(
            pr.get("title").asText(),
            pr.get("head").get("ref").asText(),
            payload.get("repository").get("full_name").asText(),
            pr.get("number").asInt(),
            pr.get("html_url").asText(),
            pr.get("user").get("login").asText(),
            pr.get("merged").asBoolean(false),
            action
        );
    }

    private void handleReview(JsonNode payload) {
        JsonNode review = payload.get("review");
        JsonNode pr = payload.get("pull_request");
        // resolve local PullRequest by repo+number, then:
        // reviewService.recordReview(localPullRequestId, reviewerUsername, state, body);
        // left as a lookup step since it needs PullRequestRepository injected here too —
        // straightforward addition following the same pattern as PrLinkingService.
    }
}

```

MODIFY the Prodhive_core main application class — add `@EnableAsync` :

```

@SpringBootApplication
@EnableAsync
public class ProdhiveCoreApplication { ... }

```

12e. Exempt the webhook path from JWT validation

MODIFY `Prodhive_gateway/.../filter/JwtAuthFilter.java` :

```

if (path.startsWith("/api/auth/") || path.startsWith("/api/core/webhooks/")) {
    return chain.filter(exchange);
}

```

This is now safe to leave unauthenticated at the gateway level because the signature check in 12b is the real security boundary — a request without a valid GitHub HMAC signature never reaches your business logic, regardless of JWT. This is the fix for the exact gap flagged (but not resolved) in the earlier generic guide.

12f. GitHub webhook setup

- 1. Repo → Settings → Webhooks → Add webhook
- 2. Payload URL: your gateway URL + `/api/core/webhooks/github`
- 3. Content type: `application/json`
- 4. **Secret:** generate a random string, put it in both GitHub's webhook config and your `GITHUB_WEBHOOK_SECRET` env var — this is what 12b verifies against
- 5. Events: "Pull requests" and "Pull request reviews"

✓ **Phase 12 done when:** a real GitHub PR event hits your endpoint, the signature check passes, resending the exact same delivery (GitHub's "Redeliver" button) is correctly ignored as a duplicate, and `WebhookDelivery` rows show PROCESSED status.

Phase 13: Advanced Features

Goal: the grab-bag phase. Treat each sub-item independently — pick what's achievable in your remaining time rather than trying to do all six.

13a. Configurable workflows (per project, not hardcoded)

```
// entity/WorkflowTransition.java
package com.prodhive_core.entity;
import jakarta.persistence.*;
import lombok.*;

@Entity
@Getter @Setter @NoArgsConstructor @AllArgsConstructor
public class WorkflowTransition {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false) private Long projectId;
    @Enumerated(EnumType.STRING) @Column(nullable = false) private IssueStatus fromStatus;
    @Enumerated(EnumType.STRING) @Column(nullable = false) private IssueStatus toStatus;
}
```

MODIFY `service/WorkflowService.java` — replace the hardcoded map with a DB-backed check, falling back to the Phase 5 defaults if a project hasn't customized anything:

```
public void validateTransition(Long projectId, IssueStatus from, IssueStatus to) {
    if (from == to) return;
    List<WorkflowTransition> custom = transitionRepository.findByProjectId(projectId);
    if (!custom.isEmpty()) {
        boolean allowed = custom.stream().anyMatch(t -> t.getFromStatus() == from && t.getToStatus() == to);
        if (!allowed) throw new InvalidTransitionException("Cannot move issue from " + from + " to " + to);
        return;
    }
    // fall back to the Phase 5 default map
    validateDefaultTransition(from, to);
}
```

Add a small `WorkflowTransitionController` so PMs can configure their project's transitions via CRUD (`POST/GET/DELETE` `/api/core/projects/{id}/workflow-transitions`).

13b. Roadmaps

MODIFY `entity/Issue.java` — add (relevant mainly for `EPIC` -type issues):

```
private LocalDate startDate;
private LocalDate dueDate;
```

```
// controller/RoadmapController.java (NEW)
@GetMapping("/api/core/projects/{projectId}/roadmap")
public ResponseEntity<List<Issue>> roadmap(@PathVariable Long projectId) {
    return ResponseEntity.ok(issueRepository.findByBoard_Project_IdAndType(projectId, IssueType.EPIC));
}
// requires: List<Issue> findByBoard_Project_IdAndType(Long projectId, IssueType type); on IssueRepository
```

Frontend renders this as a Gantt-style horizontal timeline using each epic's `startDate` / `dueDate` — a chart library like `recharts` or a small custom SVG timeline both work.

13c. Capacity planning

MODIFY `entity/Issue.java` — add:

```
private Integer storyPoints;
```

```
// entity/SprintCapacity.java (NEW)
@Entity
public class SprintCapacity {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
    @Column(nullable = false) private Long sprintId;
    @Column(nullable = false) private Long userId;
    @Column(nullable = false) private Integer capacityPoints;
}
```

```
// service method: planned vs capacity per sprint
public Map<Long, Integer> plannedPointsByUser(Long sprintId) {
    return issueRepository.findBySprintId(sprintId).stream()
        .filter(i -> i.getAssigneeId() != null && i.getStoryPoints() != null)
        .collect(Collectors.groupingBy(Issue::getAssigneeId, Collectors.summingInt(Issue::getStoryPoints)));
}
```

13d. Public API keys (alternative auth for external/API access)

```
// Prodhive_auth: entity/ApiKey.java
@Entity
public class ApiKey {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
    @Column(nullable = false) private Long organizationId;
    @Column(nullable = false, unique = true) private String keyHash; // store a hash, never the raw key
    @Column(nullable = false) private String label;
    @Column(nullable = false) private Instant createdAt = Instant.now();
    private Instant lastUsedAt;
}
```

MODIFY `Prodhive_gateway/.../filter/JwtAuthFilter.java` — add a branch: if the request carries an `X-API-Key` header instead of a Bearer token, hash it and look it up against `ApiKey` (a small internal call to Auth Service, or a shared cache) instead of decoding a JWT. Full implementation follows the same request-mutation pattern already used for JWT — omitted here for space since it's structurally identical to the existing filter, just with a different lookup.

13e. SSO — guidance, not full implementation

Real SSO (SAML or OAuth2 against Google Workspace/Azure AD/Okta) is substantial infrastructure — a registered identity provider app, certificate/metadata exchange, and careful session handling. This is reasonably out of scope for a final-year timeline. If asked in your viva: the correct answer is "we'd add `spring-boot-starter-oauth2-client` configured against an identity provider, map the provider's claims to our existing `User / OrganizationMember` model, and keep our own JWT issuance as the internal session token after the initial SSO handshake" — that's a complete, correct answer without needing the code written. Naming the right approach convincingly is worth more here than a half-working implementation.

13f. Import/export

```
// controller/ExportController.java (NEW) — CSV export of a board's issues
@GetMapping("/api/core/boards/{boardId}/export")
public void exportCsv(@PathVariable Long boardId, HttpServletResponse response) throws IOException {
    response.setContentType("text/csv");
    response.setHeader("Content-Disposition", "attachment; filename=issues.csv");
    PrintWriter writer = response.getWriter();
    writer.println("Key,Title,Status,Priority,Assignee,Sprint");
    for (Issue issue : issueRepository.findByBoardId(boardId)) {
        writer.printf("%s,%s,%s,%s,%s,%s\n",
            issue.getIssueKey(), issue.getTitle().replace(",", " "), issue.getStatus(),
            issue.getPriority(), issue.getAssigneeId(), issue.getSprint() != null ? issue.getSprint().getName() : "");
    }
}
```

```
// controller/ImportController.java (NEW) — bulk-create issues from an uploaded CSV
@PostMapping("/api/core/boards/{boardId}/import")
public ResponseEntity<List<Issue>> importCsv(@PathVariable Long boardId,
    @RequestParam("file") MultipartFile file,
    HttpServletRequest http) throws IOException {
    Long reporterId = currentUser.getUserId(http);
    List<Issue> created = new ArrayList<>();
    try (BufferedReader reader = new BufferedReader(new InputStreamReader(file.getInputStream()))) {
        reader.readLine(); // skip header
        String line;
        while ((line = reader.readLine()) != null) {
            String[] cols = line.split(",");
            IssueRequest req = new IssueRequest(cols[0], "", "TASK", "MEDIUM", boardId, null);
            created.add(issueService.create(req, reporterId));
        }
    }
    return ResponseEntity.ok(created);
}
```

✓ **Phase 13 done when:** you've picked and finished at least 2–3 of the six sub-items above (configurable workflows and roadmaps are the highest-value, most demoable choices if you can only do a subset) rather than half-finishing all six.

Summary — New Files by Service (Phases 7–13)

--	--

Service	New files
Prodhive_core	Notification, NotificationType, SavedFilter, GitHubIntegration, PullRequest, PrStatus, Review, ReviewState, WebhookDelivery, WorkflowTransition, SprintCapacity (entities) · matching repositories · NotificationService, WorkloadService, GitHubApiClient, PrLinkingService, ReviewService, GitHubWebhookProcessingService, GitHubSignatureVerifier · NotificationController, SearchController, SavedFilterController, DashboardController, WorkloadController, GitHubIntegrationController, GitHubWebhookController, ReviewController, RoadmapController, ExportController, ImportController
Prodhive_auth	ApiKey entity + repository (Phase 13d only)
Prodhive_gateway	Webhook path exemption, optional API key branch in JwtAuthFilter

This closes your full 13-phase Build Plan on the backend. Remaining work: the equivalent frontend build-out for every phase here (notifications bell + feed, search bar, dashboard widgets, GitHub connect screen, PR/review panels on the issue detail page, roadmap/capacity UI) — bring that back as a follow-up in the same file-by-file style once this backend layer is working end to end.